CS413 · MIDTERM 1

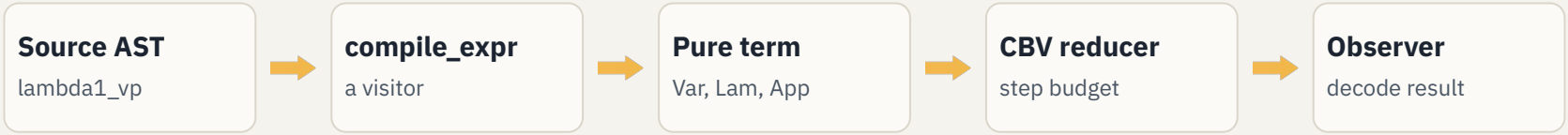
Compiling LAMBDA into Pure Lambda Calculus

Source language, Church encodings, call-by-value compilation, and results

By: Junze Ke

THE GOAL

From LAMBDA to pure lambda calculus



The compiler never evaluates. Even a looping program compiles.

The output is pure. Only variables, single-parameter lambdas, and applications.

Answers come from reduction. Python only reads the final shape.

ARCHITECTURE

One module per concern

Module	Job
target_ast.py	Var, Lam, App, printer, free variables, capture-avoiding substitution
reducer.py	Weak call-by-value reducer with a beta-value step budget
church_encodings.py	Closed Church combinators: booleans, numerals, pairs, arithmetic, Z
compiler.py	compile_expr, written as a visitor over the Assignment 5 AST
evaluator.py	evaluate_source for expected answers, evaluate_target for terms
observers.py	Decode numerals, booleans, and pairs into Python values
demo.py	Runs all 14 required examples end to end

The evaluator never imports the compiler, so evaluation cannot trigger compilation.

TARGET LANGUAGE

Terms and the reducer

Weak: never reduces under a lambda, so a lambda is already a value

Left to right: function first, then argument, then substitute

Capture-avoiding: a binder is renamed when substitution would capture it

Iterative: an explicit stack and a step budget, no Python recursion

```
Term ::= Var | Lam | App

(lam x. body) v
  ⟶ body[x := v]

v must be a lambda abstraction
```

CHURCH ENCODINGS

Data as functions

Name	Pure term
TRUE	lam t. lam f. t
FALSE	lam t. lam f. f
NUM(n)	lam s. lam z. s applied n times to z
PAIR	lam a. lam b. lam p. p a b
ADD	lam m. lam n. lam s. lam z. m s (n s z)
MUL	lam m. lam n. lam s. lam z. m (n s) z

SUB, LEQ, LT, GT, GEQ, EQ, and NEQ are built from PRED, ISZERO, and NOT. Every combinator is checked closed when the module loads.

PREDECESSOR

Shift a pair n times

n	Pair after n steps	PRED n
0	(0, 0)	0
1	(0, 1)	0
2	(1, 2)	1
3	(2, 3)	2
4	(3, 4)	3

```
STEPFN q =
  PAIR (SND q)
       (SUCC (SND q))
PRED n =
  FST (n STEPFN
       (PAIR 0 0))
```

Each step turns (a, b) into (b, b + 1), so n steps from (0, 0) give (n − 1, n).

PRED 0 = 0 needs no special case: zero steps leave the seed pair.

COMPILER

Translation rules

```
C[x] = x
C[lam x. e] = lam x. C[e]
C[e1 e2] = C[e1] C[e2]
C[let x = e1 in e2] = (lam x. C[e2]) C[e1]
C[e1 op e2] = OP C[e1] C[e2]
C[pair e1 e2] = PAIR C[e1] C[e2]
C[fix f(x). e] = Z (lam f. lam x. C[e])
```

Compositional: each rule uses only the translations of its subexpressions.

Checked everywhere: negative literals and unsupported operators raise CompileError, even in unused branches.

DELAYED BRANCHES

Only the chosen branch runs

```
C[if c then t else f]
  = (C[c] (lam u. C[t]) (lam v. C[f])) I
```

1. Wrap

Each branch becomes a lambda, so it is a value and does not run yet.

2. Select

A Church boolean returns one of the two thunks.

3. Force

Applying the chosen thunk to I runs only that branch.

`if true then 1 else LOOP` finishes in 3 steps. `(lam x. 0) LOOP` exhausts the budget, because call-by-value evaluates the argument first.

RECURSION

Z, not Y

Y loops under call-by-value

```
Y = lam g.
  (lam x. g (x x))
  (lam x. g (x x))
```

x x runs at once, so the loop starts before g can return a value.

Z waits until it is called

```
Z = lam g.
  (lam x. g (lam v. x x v))
  (lam x. g (lam v. x x v))
```

lam v. x x v is a value, so unfolding waits for the recursive call's argument.

fix f(x). e compiles to **Z (lam f. lam x. C[e])**

SCOPE

Fresh names and capture

Deterministic

One counter that never resets within a compilation, so the same input gives the same names.

Nothing reserved

Source programs may use s, z, t, f, or x. A candidate is skipped only if it is in the avoid set.

Thunk parameters

u and v are fresh against the free variables of the branch they wrap, so nothing is captured.

Scope preserved

FV(C[e]) = FV(e), including fix binders and coincident names. The tests check it.

OBSERVERS

Reading a result back

Kind	Observation context	Output
Natural	Apply to STEP = lam tail. lam c. lam z. c tail and BASE = lam c. lam z. z	Count the nodes of the spine
Boolean	Apply to two distinct closed marker lambdas	Which marker comes back
Pair	Evaluate, apply FST and SND, observe each part	A tuple, nested pairs allowed

DECODED: value read

EXHAUSTED: budget spent

MALFORMED: wrong shape

The caller supplies the expected kind, because NUM(0) and FALSE are the same term.

RESULTS

Demo outcomes

Example	Result	Exec steps	Obs steps
2 + 3 * 4	14	4	26
2 - 5	0	87	2
if 2 <= 3 then 7 else 9	7	77	9
let x = 3 in (lam y. x + y) 4	7	4	13
factorial(4)	24	471	333
fibonacci(5)	5	1,794	45
if true then 1 else LOOP	1	3	3
(lam x. 0) LOOP	exhausted	200,000	n/a
LOOP	exhausted	200,000	n/a

WRAP-UP

Testing and limits

Testing

141 unittest tests, all passing

Capture hazards, evaluation order, no reduction under lambdas, budget exhaustion

Every operator, short-circuiting, divergent arguments, nested pairs

Output uses only Var, Lam, App, and library combinators are closed

Limitations

No parser and no static type checking

No signed integers, lists, or division

Substitution rebuilds terms on every step: simple but slow

Tests do not prove the correctness claim; DESIGN.md gives an induction sketch